

Experiment – 1 a: TypeScript

Name of Student	Anushka Nilesh Shahane
Class Roll No	54
D.O.P.	23/01/25
D.O.S.	30/01/25
Sign and Grade	

Experiment – 1 a: TypeScript

1. Aim: Write a simple TypeScript program using basic data types (number, string, boolean) and operators.
2. Problem Statement :
 - a. Create a calculator in TypeScript that uses basic operations like addition, subtraction, multiplication, and division. It also gracefully handles invalid operations and division by zero..
 - b. Design a Student Result database management system using TypeScript.

// Step 1: Declare basic data types

```
const studentName: string = "John Doe";
```

```
const subject1: number = 45;
```

```
const subject2: number = 38;
```

```
const subject3: number = 50;
```

// Step 2: Calculate the average marks

```
const totalMarks: number = subject1 + subject2 + subject3;
```

```
const averageMarks: number = totalMarks / 3;
```

// Step 3: Determine if the student has passed or failed

```
const isPassed: boolean = averageMarks >= 40;
```

```
// Step 4: Display the result
console.log(Student Name: ${studentName});
console.log(Average Marks: ${averageMarks});
console.log(Result: ${isPassed ? "Passed" : "Failed"});
```

3. Theory:

1. What are the different data types in TypeScript?

TypeScript extends JavaScript by adding static types, which help catch errors early. Here are the main data types in TypeScript:

Primitive Data Types:

1. **number**: Represents all numeric values, including integers and floating-point numbers.

```
let age: number = 25;
```

2. **string**: Used for text values.

```
let name: string = "Alice";
```

3. **boolean**: Represents true or false values.

```
let isActive: boolean = true;
```

4. **undefined**: Represents a variable that is declared but not yet initialized with a value.

```
let x: undefined = undefined;
```

5. **null**: Represents an intentional absence of a value.

```
let y: null = null;
```

6. **symbol**: Represents a unique and immutable value, useful for object property keys.

```
let sym: symbol = Symbol("description");
```

Special Data Types:

1. **any**: Allows any type of value to be assigned to a variable. This bypasses TypeScript's type checking, and should be used sparingly.

```
let data: any = 42;

data = "Hello"; // This is allowed
```

2. **unknown**: Similar to **any**, but safer. You must check the type before performing operations.

```
let value: unknown = "Hello";

if (typeof value === "string") {

  console.log(value.length);

}
```

3. **void**: Used primarily to specify that a function does not return any value.

```
function logMessage(message: string): void {

  console.log(message);

}
```

4. **never**: Represents values that will never occur, like in a function that always throws an error or has an infinite loop.

```
function throwError(message: string): never {

  throw new Error(message);

}
```

Object Types:

object: A general type for any non-primitive type.

```
let obj: object = { name: "Bob", age: 30 };
```

Array: TypeScript also allows you to define arrays.

```
let numbers: number[] = [1, 2, 3];
```

2. What are Type Annotations in TypeScript?

Type annotations in TypeScript allow you to specify the type of a variable, function parameter, or return type explicitly. This helps TypeScript understand the types of variables and enables type checking, helping catch errors early during development.

Example of type annotation:

```
let age: number = 25;

let name: string = "Alice";

function greet(person: string): string {
    return "Hello, " + person;
}
```

Age is annotated as number, and name is annotated as string. The function greet expects a string parameter and returns a string.

3. How do you compile TypeScript files?

To compile TypeScript files into JavaScript, you need the TypeScript compiler (tsc). You can compile TypeScript files using the following steps:

Install TypeScript (if not already installed):

```
npm install -g typescript
```

1. Compile TypeScript file: To compile a TypeScript file (e.g., app.ts) into JavaScript, run:

```
tsc app.ts
```

2. This will generate an app.js file in the same directory.

Compile multiple files: If you want to compile all .ts files in a project, you can run:

```
tsc
```

3. TypeScript will automatically use the tsconfig.json file (if present) to configure the compilation process.

Automatically watch for changes: You can also use the --watch flag to automatically recompile your code whenever you make changes:

```
tsc --watch
```

1.

4. What is the difference between JavaScript and TypeScript?

While JavaScript is a dynamically typed language, TypeScript is a statically typed superset of JavaScript. Here are the main differences:

1. Static vs Dynamic Typing:
 - a. JavaScript: Types are determined at runtime.
 - b. TypeScript: Types are explicitly declared or inferred at compile-time.
2. Type Annotations:
 - a. JavaScript: Does not have type annotations.
 - b. TypeScript: You can add type annotations to variables, function parameters, and return types.
3. Compilation:

- a. JavaScript: It is directly executed by the browser or Node.js without the need for a compilation step.
 - b. TypeScript: Needs to be compiled into JavaScript before it can be run in a browser or Node.js.
- 4. Error Checking:
 - a. JavaScript: Errors are only detected at runtime.
 - b. TypeScript: Errors are caught during compilation, helping developers catch bugs earlier.
- 5. Advanced Features:
 - a. TypeScript provides features like interfaces, generics, enums, etc., which are not available in JavaScript.

5. Compare how JavaScript and TypeScript implement Inheritance.

Both JavaScript and TypeScript support inheritance, but TypeScript allows for stronger typing, making it easier to work with classes and inheritance.

JavaScript Example:

```
class Animal {  
  
    constructor(name) {  
  
        this.name = name;  
  
    }  
  
    speak() {  
  
        console.log(`${this.name} makes a sound`);  
  
    }  
  
}  
  
class Dog extends Animal {
```

```
    speak() {  
        console.log(`${this.name} barks`);  
    }  
}
```

```
const dog = new Dog("Buddy");  
dog.speak(); // Output: Buddy barks
```

TypeScript Example:

```
class Animal {  
    constructor(public name: string) {}  
  
    speak(): void {  
        console.log(`${this.name} makes a sound`);  
    }  
}
```

```
class Dog extends Animal {  
    speak(): void {  
        console.log(`${this.name} barks`);  
    }  
}
```

```
const dog = new Dog("Buddy");  
  
dog.speak(); // Output: Buddy barks
```

In TypeScript, we can add explicit types to class properties and method signatures. TypeScript also provides better type-checking and prevents errors like accidentally passing the wrong type.

6. How do generics make the code flexible, and why should we use generics over other types?

Generics in TypeScript allow you to write flexible and reusable code. Instead of hard-coding specific types, you can write functions or classes that work with any type while still maintaining type safety.

Why generics are useful:

Type Safety: With generics, you maintain type safety, ensuring that values passed into functions or classes are of the correct type.

Reusability: Generics allow you to write a single function or class that can work with multiple types, reducing code duplication.

Example with Generics:

```
function identity<T>(value: T): T {  
  
    return value;  
  
}  
  
let result = identity(5);  
  
let result2 = identity("Hello");
```


Generics are better than any because any allows any value to be passed in, leading to potential runtime errors, whereas generics retain type safety.

In Lab Assignment 3, using generics over any ensures that the function can handle different data types while still being type-safe. If you used any, you wouldn't get the benefits of TypeScript's static type checking, which could lead to errors when working with unexpected types.

7. What is the difference between Classes and Interfaces in TypeScript? Where are interfaces used?

Classes:

- A class is a blueprint for creating objects with specific properties and methods.
- Classes can have implementations (method bodies), and can include features like constructors and inheritance.

Interfaces:

- An interface defines the shape of an object but does not provide implementation.
- It defines properties and methods that a class or object must implement, but it does not define how they should be implemented.

Difference:

- Classes can include both definitions and implementations, whereas interfaces only define the structure without implementation.

Example of Class and Interface:

```
interface Animal {  
    name: string;  
    speak(): void;  
}
```

```
class Dog implements Animal {  
    constructor(public name: string) {}  
  
    speak() {  
        console.log(`${this.name} barks`);  
    }  
}  
  
const dog = new Dog("Buddy");  
  
dog.speak(); // Output: Buddy barks
```

Where are Interfaces Used?

1. Interfaces are commonly used to define contracts or shapes for objects, ensuring that classes or objects adhere to a specific structure.
2. They can also be used to define the structure for function types, array types, or any other custom type.

4. Output:

CODE:

```
import * as readline from 'readline';

class Calculator {

  add(a: string | number, b: string | number): string | number {

    if (typeof a === 'string' || typeof b === 'string') {

      return `${a}${b}`;

    }

    return a + b; // Perform addition if both are numbers

  }

  subtract(a: number, b: number): number {

    return a - b;

  }

  multiply(a: number, b: number): number {

    return a * b;

  }

  divide(a: number, b: number): number | string {

    if (b === 0) {

      return "Error: Division by zero is not allowed.";

    }

    return a / b;

  }

}
```

```
operate(a: string | number, b: string | number, operation: string): string | number | never {  
  switch (operation) {  
    case 'add':  
      return this.add(a, b);  
    case 'subtract':  
      if (typeof a === 'string' || typeof b === 'string') {  
        throw new Error("Cannot subtract strings.");  
      }  
      return this.subtract(a as number, b as number);  
    case 'multiply':  
      if (typeof a === 'string' || typeof b === 'string') {  
        throw new Error("Cannot multiply strings.");  
      }  
      return this.multiply(a as number, b as number);  
    case 'divide':  
      if (typeof a === 'string' || typeof b === 'string') {  
        throw new Error("Cannot divide strings.");  
      }  
      return this.divide(a as number, b as number);  
    default:  
      throw new Error('Error: Invalid operation.');
```

```
}
```

```
}
```

```
const rl = readline.createInterface({
```

```
  input: process.stdin,
```

```
  output: process.stdout
```

```
});
```

```
const askQuestion = (question: string): Promise<string> => {
```

```
  return new Promise((resolve) => rl.question(question, resolve));
```

```
};
```

```
const main = async () => {
```

```
  try {
```

```
    const input1 = await askQuestion("Enter the first value: ");
```

```
    const input2 = await askQuestion("Enter the second value: ");
```

```
    const operation = await askQuestion("Enter operation (add, subtract, multiply, divide): ");
```

```
    const num1 = parseFloat(input1);
```

```
    const num2 = parseFloat(input2);
```

```
    const calculator = new Calculator();
```

```
    const result = calculator.operate(
```

```
      isNaN(num1) ? input1 : num1,
```

```
      isNaN(num2) ? input2 : num2,
```

```
      operation
```

```
    );
```

```
console.log(`Result: ${result}`);

} catch (error) {

console.error(error.message);

} finally {

rl.close();

}

};

main();
```

OUTPUT :

```
C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>tsc Calculator.ts

C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first number: 32
Enter the second number: 22
Enter operation (add, subtract, multiply, divide): add
Result: 54
```

```
C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first number: 33
Enter the second number: 44
Enter operation (add, subtract, multiply, divide): add
Result: 77

C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first number: 22
Enter the second number: 0
Enter operation (add, subtract, multiply, divide): divide
Result: Error: Division by zero is not allowed.

C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first number: 33
Enter the second number: 8
Enter operation (add, subtract, multiply, divide): subtract
Result: 25

C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first number: 8
Enter the second number: 5
Enter operation (add, subtract, multiply, divide): multiply
Result: 40
```

```
C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first value: Anushka
Enter the second value: Shahane
Enter operation (add, subtract, multiply, divide): add
Result: Anushka Shahane

C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node Calculator.js
Enter the first value: 44
Enter the second value: 2
Enter operation (add, subtract, multiply, divide): square
Error: Invalid operation.
```

B] Design a Student Result database management system using TypeScript.

// Step 1: Declare basic data types

```
const studentName: string = "John Doe";
```

```
const subject1: number = 45;
```

```
const subject2: number = 38;
```

```
const subject3: number = 50;
```

// Step 2: Calculate the average marks

```
const totalMarks: number = subject1 + subject2 + subject3;
```

```
const averageMarks: number = totalMarks / 3;
```

// Step 3: Determine if the student has passed or failed

```
const isPassed: boolean = averageMarks >= 40;
```

// Step 4: Display the result

```
console.log(Student Name: ${studentName});
```

```
console.log(Average Marks: ${averageMarks});
```

```
console.log(Result: ${isPassed ? "Passed" : "Failed"});
```

CODE:

```
const studentName: string = "Anushka Shahane";
```

```
const subject1: number = 45;
```

```
const subject2: number = 38;
```

```
const subject3: number = 50;

// Step 2: Calculate the average marks

const totalMarks: number = subject1 + subject2 + subject3;

const averageMarks: number = totalMarks / 3;


const isPassed: boolean = averageMarks >= 40;


console.log(`Student Name: ${studentName}`);

console.log(`Average Marks: ${averageMarks}`);

console.log(`Result: ${isPassed ? "Passed" : "Failed"}`);
```

OUTPUT :

```
C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>tsc student.ts

C:\Users\Student.DESKTOP-HH2106Q\Desktop\folder>node student.js
Student Name: Anushka Shahane
Average Marks: 44.333333333333336
Result: Passed
```

Github link : https://github.com/Anushka3204/WebX_Experiment_1

Conclusion : In this experiment, I successfully explored the foundational features of TypeScript, including data types, type annotations, and object-oriented programming concepts. By implementing a calculator using basic operations and handling errors such as invalid operations and division by zero, we understood how TypeScript ensures type safety and improves code readability and maintainability.